

60-Day Interview Challenge

Complete Preparation Guide — Fresher Track (Aggressive Mode)

Python Fullstack

AI / ML Engineer

Data Engineer

Data Scientist

Software Tester

SDE

60

Days

600+

Interview Qs

600+

Python Coding

600+

SQL Problems

D

Prepared for: Preetham | MCA Graduate — VVIET Mysuru | 2026

Target: Bengaluru Companies — AI/ML · Python · Data Engineering · Testing

Each day: 10 Interview Questions · 10 Python Coding · 10 SQL · 3 DSA Problems

Complete previous day to unlock next day. Day-locked progression system.

Table of Contents

Week 1 (Days 1–5) — Python Basics, OOP, OS/Networks, System Design, Web Frameworks

Week 2 (Days 6–10) — ML Foundations, Deep Learning, Data Engineering, Cloud/DevOps, Testing

Week 3 (Days 11–15) — NLP/LLMs, Advanced Python, Python Basics II, OOP II, OS/Networks II

Week 4–6 (Days 16–30) — Intermediate rotations with harder variants of all topics

Week 7–9 (Days 31–45) — Advanced System Design, ML at scale, Cloud Architecture, DSA mastery

Week 10–12 (Days 46–60) — Mock interview simulations, full-stack deep dives, competitive coding

Each day contains 4 sections:

Section A: 10 Interview Questions with detailed answers

Section B: 10 Python Coding problems with solutions and code

Section C: 10 SQL Coding problems with queries and explanations

Section D: 3 DSA Problems with full implementations

Day 1 — Python Fundamentals

Python Basics · DSA: Arrays & Lists

Topics: Python · CS Fundamentals · OS Basics

Section A — Interview Questions

Q1. What are Python's key features that make it suitable for data science and web development?

● **HARD**

Answer:

Python offers dynamic typing, concise readable syntax, and a rich ecosystem (NumPy, pandas, scikit-learn, Django, FastAPI). Its interpreted nature enables rapid prototyping. The GIL simplifies memory management for single-threaded I/O-bound tasks while libraries like multiprocessing bypass it for CPU-bound work.

Code:

```
import numpy as np
import pandas as pd
# Vectorised ops – no loops needed
data = np.array([1,2,3,4,5])
print(data.mean(), data.std())
```

Q2. Explain mutable vs immutable objects in Python with examples.

● **HARD**

Answer:

Immutable objects (int, float, str, tuple, frozenset) cannot be changed after creation — rebinding creates a new object. Mutable objects (list, dict, set, bytearray) can be modified in-place. This matters for function arguments (mutable defaults are shared), caching, and hashing (only immutable objects are hashable).

Code:

```
# Immutable: str
s = 'hello'
id1 = id(s)
s += ' world' # new object
print(id(s) == id1) # False

# Mutable: list
lst = [1,2,3]
id2 = id(lst)
lst.append(4) # same object
print(id(lst) == id2) # True
```

Q3. What is the difference between list, tuple, set, and dict? When to use each?

● **HARD**

Answer:

list: ordered, mutable, allows duplicates — use for ordered collections you modify. tuple: ordered, immutable, hashable — use for fixed records, dict keys, return values. set: unordered, unique elements, O(1) lookup — use for membership testing, deduplication. dict: key-value mapping, O(1) get/set — use whenever you need label-based lookup.

Code:

```
lst = [1,2,2,3] # ordered, duplicates OK
tup = (1,'a', 3.14) # immutable record
s = {1,2,2,3} # {1,2,3} – deduped
d = {'name': 'Preetham', 'role': 'SDE'}
print(d['name']) # O(1) lookup
```

Q4. Explain Python's memory management and garbage collection.

● **MED**

Answer:

CPython uses reference counting: every object tracks how many references point to it. When the count hits zero, memory is freed immediately. A cyclic garbage collector (gc module) handles reference cycles that reference counting misses. Memory is allocated via a private heap; the allocator uses pools and arenas for small objects.

Code:

```
import sys, gc
a = []
print(sys.getrefcount(a)) # 2 (a + getrefcount arg)
b = a
print(sys.getrefcount(a)) # 3
del b
gc.collect() # force cycle collection
```

Q5. What are *args and **kwargs? Demonstrate with examples.

● **HARD**

Answer:

*args collects positional arguments into a tuple; **kwargs collects keyword arguments into a dict. They allow functions to accept arbitrary numbers of arguments. Order must be: regular → *args → keyword-only → **kwargs.

Code:

```
def describe(*args, sep=' ', **kwargs):
    print(sep.join(str(a) for a in args))
    for k,v in kwargs.items():
        print(f'{k}={v}')
describe(1,2,3, sep='-', role='SDE', level='L3')
# Output: 1-2-3 \n role=SDE \n level=L3
```

Q6. What is the difference between '==' and 'is' in Python?

● MED

Answer:

== tests value equality (calls `__eq__`). is tests identity — whether two names point to the same object in memory (same `id()`). CPython interns small integers (-5 to 256) and short strings, so 'is' can misleadingly return True; never rely on 'is' for value comparison.

Code:

```
a = [1,2,3]
b = [1,2,3]
print(a == b) # True – same values
print(a is b) # False – different objects

x = 256; y = 256
print(x is y) # True – interned int
```

Q7. Explain list comprehensions vs generator expressions. Memory implications?

● HARD

Answer:

List comprehensions evaluate eagerly and store all results in RAM — $O(n)$ memory. Generator expressions are lazy: they yield one item at a time — $O(1)$ memory. Use generators for large or infinite sequences where you only need to iterate once.

Code:

```
import sys
lst = [x*x for x in range(10_000)] # list
gen = (x*x for x in range(10_000)) # generator
print(sys.getsizeof(lst)) # ~87,624 bytes
print(sys.getsizeof(gen)) # ~112 bytes
```

Q8. What are Python decorators? Write a timing decorator.

● HARD

Answer:

A decorator is a callable that wraps another callable to extend its behaviour without modifying its source. They are syntactic sugar for `func = decorator(func)`. Common uses: logging, timing, caching, auth, validation.

Code:

```
import time, functools
def timeit(fn):
    @functools.wraps(fn)
    def wrapper(*a, **kw):
        t0 = time.perf_counter()
        result = fn(*a, **kw)
        print(f'{fn.__name__} took {time.perf_counter()-t0:.4f}s')
        return result
    return wrapper
@timeit
```

```
def slow():
    time.sleep(0.5)
slow() # slow took 0.5001s
```

Q9. Explain the GIL (Global Interpreter Lock) and its impact on multithreading.

● **HARD**

Answer:

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This protects reference counts but means threads cannot run Python code in parallel — CPU-bound multithreading doesn't scale. I/O-bound threads are fine because the GIL is released during I/O. Use multiprocessing or concurrent.futures.ProcessPoolExecutor for CPU parallelism.

Code:

```
# CPU-bound: use processes, not threads
from concurrent.futures import ProcessPoolExecutor

def cpu_task(n):
    return sum(i*i for i in range(n))

with ProcessPoolExecutor() as ex:
    results = list(ex.map(cpu_task, [10**6]*4))
```

Q10. What is the difference between deep copy and shallow copy?

● **MED**

Answer:

Shallow copy (copy.copy or list[:]) creates a new outer object but its elements still reference the same inner objects. Deep copy (copy.deepcopy) recursively copies everything — all nested objects are independent. Use shallow copy when elements are immutable; deep copy when you need full isolation.

Code:

```
import copy
orig = [[1,2],[3,4]]
shallow = copy.copy(orig)
deep = copy.deepcopy(orig)
orig[0].append(99)
print(shallow[0]) # [1, 2, 99] – shared inner list
print(deep[0]) # [1, 2] – independent
```

Section B — Python Coding Problems

Q1. Two Sum: Given array and target, return indices of two numbers that add up to target.

● **MED**

Answer:

Use a hash map to store each number's index as you iterate. For each num, check if (target - num) is already in the map.

Hint: Hash map for $O(n)$

Code:

```
def two_sum(nums, target):
```

```
seen = {} # val -> index
for i, n in enumerate(nums):
    need = target - n
    if need in seen:
        return [seen[need], i]
    seen[n] = i
return []

print(two_sum([2,7,11,15], 9)) # [0,1]
```

Q2. Reverse a string without using slicing.

● EASY

Answer:

Convert to list, use two pointers swapping from both ends toward the middle.

Hint: Two pointer approach

Code:

```
def reverse_str(s):
    chars = list(s)
    l, r = 0, len(chars)-1
    while l < r:
        chars[l], chars[r] = chars[r], chars[l]
        l += 1; r -= 1
    return ''.join(chars)

print(reverse_str('Preetham')) # mahteert
```

Q3. Find all duplicates in a list using $O(n)$ time and $O(1)$ extra space.

● HARD

Answer:

For each element, use its absolute value as an index and negate the value at that index. If it's already negative, it's a duplicate.

Hint: Negative marking on array values

Code:

```
def find_duplicates(nums):
    res = []
    for n in nums:
        idx = abs(n) - 1
        if nums[idx] < 0:
            res.append(abs(n))
        else:
            nums[idx] = -nums[idx]
    return res

print(find_duplicates([4,3,2,7,8,2,3,1])) # [2,3]
```

Q4. Flatten a nested list of arbitrary depth.

● **HARD**

Answer:

Use recursion: if element is a list, recurse; else yield the element.

Hint: Recursion or stack

Code:

```
def flatten(lst):
    for item in lst:
        if isinstance(item, list):
            yield from flatten(item)
        else:
            yield item
print(list(flatten([1, [2, [3, [4]], 5]]))) # [1, 2, 3, 4, 5]
```

Q5. Implement a stack using two queues.

● **HARD**

Answer:

After each push, rotate the queue so the newest element is at the front, making pop O(1).

Hint: Push O(n) — rotate queue after push

Code:

```
from collections import deque
class Stack:
    def __init__(self):
        self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q)-1):
            self.q.append(self.q.popleft())
    def pop(self): return self.q.popleft()
    def top(self): return self.q[0]
    def empty(self): return not self.q
s = Stack(); s.push(1); s.push(2)
print(s.top()) # 2
```

Q6. Count frequency of each character in a string without Counter.

● **EASY**

Answer:

Iterate characters; use dict.get with default 0.

Hint: Plain dict approach

Code:

```
def char_freq(s):
    freq = {}
    for c in s:
        freq[c] = freq.get(c, 0) + 1
```



```
return freq
print(char_freq('preetham'))
```

Q7. Find the longest common prefix from a list of strings.

● MED

Answer:

Sort the list. The longest common prefix can only exist between the first and last strings after sorting.

Hint: Sort and compare first/last only

Code:

```
def longest_prefix(strs):
    if not strs: return ''
    strs.sort()
    first, last = strs[0], strs[-1]
    i = 0
    while i < len(first) and i < len(last) and first[i]==last[i]:
        i += 1
    return first[:i]
print(longest_prefix(['flower', 'flow', 'flight'])) # 'fl'
```

Q8. Implement binary search on a sorted list.

● MED

Answer:

Maintain lo and hi pointers; compare mid element and shrink window.

Hint: lo, hi, mid loop

Code:

```
def binary_search(arr, target):
    lo, hi = 0, len(arr)-1
    while lo <= hi:
        mid = (lo+hi)//2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid+1
        else: hi = mid-1
    return -1
print(binary_search([1,3,5,7,9,11], 7)) # 3
```

Q9. Check if a string is a palindrome ignoring spaces and case.

● EASY

Answer:

Filter only alphanumeric chars, lowercase, then two-pointer check.

Hint: Two pointers after cleaning

Code:

```
def is_palindrome(s):
    clean = [c.lower() for c in s if c.isalnum()]
    l, r = 0, len(clean)-1
```

```

while l < r:
    if clean[l] != clean[r]: return False
    l+=1; r-=1
return True
print(is_palindrome('A man a plan a canal Panama')) # True

```

Q10. Find the maximum subarray sum (Kadane's algorithm).

● **HARD**

Answer:

Extend current subarray if positive, else start fresh. Track global maximum.

Hint: Track current_sum and global max

Code:

```

def max_subarray(nums):
    cur = best = nums[0]
    for n in nums[1:]:
        cur = max(n, cur+n)
        best = max(best, cur)
    return best
print(max_subarray([-2,1, -3,4, -1,2,1, -5,4])) # 6

```

Section C — SQL Coding Problems

Q1. Write a query to find the 2nd highest salary from an Employees table.

● **HARD**

Answer:

Use a subquery to exclude the MAX salary, then find MAX of the remaining. Handles ties correctly.

Code:

```

SELECT MAX(salary) AS second_highest
FROM Employees
WHERE salary < (SELECT MAX(salary) FROM Employees);

-- Alternative using DENSE_RANK:
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) rk
    FROM Employees
) t WHERE rk = 2;

```

Q2. Explain the difference between WHERE and HAVING clauses.

● **MED**

Answer:

WHERE filters rows BEFORE aggregation (cannot use aggregate functions). HAVING filters groups AFTER GROUP BY aggregation (can use aggregate functions like COUNT, SUM).

Code:

```

-- WHERE: filter rows before grouping

```

```
SELECT dept, COUNT(*) cnt
FROM Employees
WHERE salary > 50000 -- row-level filter
GROUP BY dept
HAVING COUNT(*) > 5; -- group-level filter
```

Q3. Write a query to find duplicate rows in a table.

● **HARD**

Answer:

Group by all columns you consider part of the key; HAVING COUNT(*) > 1 reveals duplicates.

Code:

```
SELECT email, COUNT(*) cnt
FROM Users
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY cnt DESC;

-- To see full duplicate rows:
SELECT * FROM Users
WHERE email IN (
SELECT email FROM Users GROUP BY email HAVING COUNT(*)>1
);
```

Q4. What are the different types of JOINS? Write examples for each.

● **HARD**

Answer:

INNER JOIN: matching rows only. LEFT JOIN: all left + matching right (NULLs for non-matches). RIGHT JOIN: all right + matching left. FULL OUTER JOIN: all rows from both sides. CROSS JOIN: cartesian product. SELF JOIN: table joined to itself.

Code:

```
-- INNER
SELECT e.name, d.name FROM Employees e
INNER JOIN Departments d ON e.dept_id=d.id;

-- LEFT
SELECT e.name, d.name FROM Employees e
LEFT JOIN Departments d ON e.dept_id=d.id;

-- SELF JOIN (employee & manager)
SELECT e.name, m.name AS manager
FROM Employees e LEFT JOIN Employees m ON e.mgr_id=m.id;
```

Q5. Find all employees who earn more than the average salary.

● **MED**

Answer:

Scalar subquery computes the average once; WHERE filters against it.

Code:

```
SELECT name, salary
```

```
FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees)
ORDER BY salary DESC;
```

Q6. What is a PRIMARY KEY vs UNIQUE KEY vs FOREIGN KEY?

● MED

Answer:

PRIMARY KEY: uniquely identifies every row, NOT NULL, one per table. UNIQUE KEY: enforces uniqueness, allows one NULL, multiple per table. FOREIGN KEY: references PRIMARY/UNIQUE key in another table, enforces referential integrity.

Code:

```
CREATE TABLE Orders (
order_id INT PRIMARY KEY,
order_ref VARCHAR(20) UNIQUE,
customer_id INT NOT NULL,
FOREIGN KEY (customer_id) REFERENCES Customers(id)
ON DELETE CASCADE
);
```

Q7. Write a query to get the top 3 departments by employee count.

● HARD

Answer:

Group by department, count employees, order descending, limit to 3.

Code:

```
SELECT d.name, COUNT(e.id) AS headcount
FROM Departments d
LEFT JOIN Employees e ON e.dept_id = d.id
GROUP BY d.name
ORDER BY headcount DESC
LIMIT 3;
```

Q8. What is the difference between TRUNCATE, DELETE, and DROP?

● MED

Answer:

DELETE: DML, removes rows with WHERE filter, logged, can rollback, triggers fire. TRUNCATE: DDL, removes all rows fast, minimal logging, resets identity, cannot rollback in most DBs. DROP: DDL, removes entire table structure and data permanently.

Code:

```
DELETE FROM logs WHERE created_at < '2024-01-01'; -- selective, slow
TRUNCATE TABLE logs; -- fast, all rows gone
DROP TABLE logs; -- table & schema gone
```

Q9. Find customers who have placed orders in every month of 2024.

● HARD

Answer:

Count distinct months per customer where year=2024; keep those with 12 distinct months.

Code:

```
SELECT customer_id
FROM Orders
WHERE YEAR(order_date) = 2024
GROUP BY customer_id
HAVING COUNT(DISTINCT MONTH(order_date)) = 12;
```

Q10. What are indexes and how do they improve query performance?

● HARD

Answer:

An index is a separate data structure (usually B-Tree) that maps column values to row locations, avoiding full table scans. Trade-off: faster reads, slower writes (INSERT/UPDATE/DELETE must maintain the index). Types: B-Tree (default), Hash, GIN/GiST (PostgreSQL full-text), Bitmap.

Code:

```
-- Create
CREATE INDEX idx_emp_dept ON Employees(dept_id);
-- Partial index (only active employees)
CREATE INDEX idx_active_emp ON Employees(dept_id)
WHERE status='active';
-- Check usage
EXPLAIN ANALYZE SELECT * FROM Employees WHERE dept_id=5;
```

Section D — DSA Problems

Q1. Implement a singly linked list with insert, delete, and reverse operations.

● HARD

Answer:

Define Node with val and next. Insert at head O(1); delete scans for prev node O(n); reverse iteratively in O(n).

Hint: Node class + head pointer

Code:

```
class Node:
def __init__(self, val):
self.val = val; self.next = None

class LinkedList:
def __init__(self): self.head = None
def insert(self, val):
n = Node(val); n.next = self.head; self.head = n
def delete(self, val):
```

```

cur = self.head; prev = None
while cur:
    if cur.val==val:
    if prev: prev.next=cur.next
    else: self.head=cur.next
    return
    prev,cur = cur,cur.next
def reverse(self):
    prev,cur = None, self.head
    while cur:
        nxt=cur.next; cur.next=prev; prev=cur; cur=nxt
    self.head=prev
def __repr__(self):
    vals=[]; cur=self.head
    while cur: vals.append(str(cur.val)); cur=cur.next
    return '->'.join(vals)

```

Q2. Find the middle of a linked list in one pass.

● MED

Answer:

Use two pointers: slow moves 1 step, fast moves 2 steps. When fast reaches the end, slow is at the middle.

Hint: Slow/fast pointer (Floyd's tortoise)

Code:

```

def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow # slow.val is the middle

```

Q3. Detect a cycle in a linked list (Floyd's algorithm).

● HARD

Answer:

Two pointers at different speeds will meet inside a cycle. If fast reaches None, no cycle.

Hint: Slow/fast pointers — if they meet, cycle exists

Code:

```

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False

```

Day 2 — OOP & Design Patterns

OOP + DBMS · DSA: Linked Lists

Topics: OOP · DBMS · Patterns

Section A — Interview Questions

Q1. Explain the 4 pillars of OOP with real-world examples.

● **HARD**

Answer:

Encapsulation: hiding internal state (BankAccount hides balance). Abstraction: exposing only what's necessary (Shape.area() without implementation). Inheritance: reusing behaviour (Dog extends Animal). Polymorphism: same interface, different behaviour (duck typing in Python).

Code:

```
class Animal:
    def speak(self): raise NotImplementedError

class Dog(Animal):
    def speak(self): return 'Woof'

class Cat(Animal):
    def speak(self): return 'Meow'

for a in [Dog(), Cat()]:
    print(a.speak()) # Polymorphism
```

Q2. What is the difference between abstraction and encapsulation?

● **HARD**

Answer:

Abstraction hides complexity by exposing only relevant behaviour through interfaces/abstract classes — WHAT an object does. Encapsulation hides internal state/implementation using access control (private attributes) — HOW it does it. Both reduce coupling, but they operate at different levels.

Code:

```
from abc import ABC, abstractmethod

class Shape(ABC): # Abstraction
    @abstractmethod
    def area(self): ...

class Circle(Shape): # Encapsulation
    def __init__(self, r):
        self.__r = r # private
    def area(self):
        return 3.14*self.__r**2
```

Q3. Explain method overloading vs method overriding in Python.

● HARD

Answer:

Python does not support traditional overloading (no multiple signatures for same name — last definition wins). Simulate with default args or *args. Method overriding: subclass provides its own implementation of a parent method. Use `super()` to call the parent version.

Code:

```
class Base:
    def greet(self, name='World'):
        return f'Hello {name}'

class Child(Base):
    def greet(self, name='Preetham'): # Override
        return super().greet(name).upper()

print(Child().greet()) # HELLO PREETHAM
```

Q4. What are class methods, static methods, and instance methods?

● HARD

Answer:

Instance method receives `self` — accesses instance and class. Class method receives `cls` (`@classmethod`) — accesses class, not instance; used for alternate constructors. Static method (`@staticmethod`) receives neither — utility function namespaced to the class.

Code:

```
class Date:
    def __init__(self, y, m, d):
        self.y, self.m, self.d = y, m, d

    @classmethod
    def from_string(cls, s): # alt constructor
        return cls(*map(int, s.split('-')))

    @staticmethod
    def is_leap(year): # utility
        return year%4==0 and (year%100!=0 or year%400==0)

print(Date.from_string('2024-08-15').y)
print(Date.is_leap(2024))
```

Q5. Explain MRO (Method Resolution Order) in Python's multiple inheritance.

● HARD

Answer:

Python uses the C3 Linearisation algorithm to determine MRO. It guarantees that a class always appears before its parents and the left parent before the right. Use `ClassName.__mro__` or `mro()` to inspect. The 'Diamond Problem' is resolved deterministically.

Code:

```
class A: def who(self): return 'A'
class B(A): def who(self): return 'B'
class C(A): def who(self): return 'C'
```



```
class D(B,C): pass
print(D.__mro__)
# D -> B -> C -> A -> object
print(D().who()) # 'B'
```

Q6. What are SOLID principles? Explain each with Python examples.

● **HARD**

Answer:

S: Single Responsibility — one class, one reason to change. O: Open/Closed — open for extension, closed for modification (use inheritance/strategy). L: Liskov Substitution — subclass must be substitutable for parent. I: Interface Segregation — many specific interfaces > one general. D: Dependency Inversion — depend on abstractions not concretions.

Code:

```
# Open/Closed example
from abc import ABC, abstractmethod
class Discount(ABC):
    @abstractmethod
    def apply(self, price): ...
class SeasonalDiscount(Discount):
    def apply(self, price): return price * 0.9
class NoDiscount(Discount):
    def apply(self, price): return price
# New discount type: add a class, change nothing else
```

Q7. What is the difference between composition and inheritance?

● **HARD**

Answer:

Inheritance (IS-A): Dog IS-A Animal. Shares code via parent class; tight coupling. Composition (HAS-A): Car HAS-A Engine. Objects collaborate via references; looser coupling. Prefer composition — it avoids fragile base class problem and enables runtime flexibility.

Code:

```
# Composition > Inheritance for behaviour reuse
class Engine:
    def start(self): return 'vroom'
class Car:
    def __init__(self):
        self.engine = Engine() # HAS-A
    def drive(self):
        return self.engine.start()
```

Q8. Explain the Singleton design pattern. Implement thread-safe version.

● HARD

Answer:

Singleton ensures only one instance of a class exists. Use a class variable and override `__new__`. For thread safety, use a `threading.Lock` with double-checked locking.

Code:

```
import threading

class Singleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None: # double-check
                    cls._instance = super().__new__(cls)
            return cls._instance

a = Singleton(); b = Singleton()
print(a is b) # True
```

Q9. What are magic/dunder methods in Python? Give 5 examples.

● HARD

Answer:

Magic methods are double-underscore methods that Python calls implicitly. `__init__`: constructor. `__repr__`/`__str__`: string representation. `__len__`: `len()`. `__getitem__`: indexing `[]`. `__eq__`/`__lt__`: comparisons. `__enter__`/`__exit__`: context managers. `__call__`: make instance callable.

Code:

```
class Vector:
    def __init__(self, x, y): self.x, self.y = x, y
    def __repr__(self): return f'Vector({self.x},{self.y})'
    def __add__(self, o): return Vector(self.x+o.x, self.y+o.y)
    def __len__(self): return 2
    def __eq__(self, o): return self.x==o.x and self.y==o.y

v = Vector(1,2)+Vector(3,4)
print(v) # Vector(4,6)
```

Q10. What is ACID in databases? Give real-world examples for each property.

● HARD

Answer:

Atomicity: all operations in a transaction succeed or all are rolled back (bank transfer — debit and credit both happen or neither). Consistency: transaction brings DB from one valid state to another (balance never goes negative). Isolation: concurrent transactions don't interfere (two users booking the last seat). Durability: committed data survives crashes (confirmed order persists through server restart).

Code:

```
-- Bank transfer transaction
```

```
BEGIN;
UPDATE accounts SET balance=balance-500 WHERE id=1;
UPDATE accounts SET balance=balance+500 WHERE id=2;
-- If any step fails, ROLLBACK ensures atomicity
COMMIT;
```

Section B — Python Coding Problems

Q1. Implement LRU Cache with $O(1)$ get/put.

● **HARD**

Answer:

Use Python's OrderedDict: move_to_end marks recent use; popitem(last=False) evicts LRU.

Hint: OrderedDict or doubly-linked list + hashmap

Code:

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, cap):
        self.cap = cap
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, val):
        if key in self.cache:
            self.cache.move_to_end(key)
            self.cache[key] = val
        if len(self.cache) > self.cap:
            self.cache.popitem(last=False)

c = LRUCache(2); c.put(1,1); c.put(2,2)
print(c.get(1)) # 1
c.put(3,3) # evicts key 2
print(c.get(2)) # -1
```

Q2. Merge two sorted linked lists.

● **MED**

Answer:

Use a dummy node to avoid edge cases. Compare heads, attach smaller, advance pointer.

Hint: Two pointer merge with dummy head

Code:

```
def merge_lists(l1, l2):
    dummy = Node(0); cur = dummy
    while l1 and l2:
        if l1.val <= l2.val:
```

```

cur.next = l1; l1 = l1.next
else:
cur.next = l2; l2 = l2.next
cur = cur.next
cur.next = l1 or l2
return dummy.next

```

Q3. Find all permutations of a string.

● **HARD**

Answer:

Fix one character, recurse on the rest. Use a visited array to handle duplicates.

Hint: Backtracking with used-set

Code:

```

def permutations(s):
    res = []
    def bt(path, used):
        if len(path)==len(s): res.append(''.join(path)); return
        for i,c in enumerate(s):
            if used[i]: continue
            used[i]=True; path.append(c)
            bt(path, used)
            path.pop(); used[i]=False
        bt([], [False]*len(s))
    return res
print(permutations('abc'))

```

Q4. Given a BST, find the kth smallest element.

● **HARD**

Answer:

Inorder traversal of BST gives sorted sequence. Count nodes visited; return when count == k.

Hint: Inorder traversal (sorted order)

Code:

```

def kth_smallest(root, k):
    stack, count = [], 0
    cur = root
    while cur or stack:
        while cur: stack.append(cur); cur=cur.left
        cur = stack.pop(); count+=1
        if count==k: return cur.val
        cur = cur.right

```

Q5. Implement a queue using only stacks.

● **HARD**

Answer:

Push to inbox. On dequeue, if outbox empty, reverse inbox into outbox.

Hint: Two stacks: inbox and outbox

Code:

```
class Queue:
def __init__(self):
self.inbox=[]; self.outbox=[]
def enqueue(self, x): self.inbox.append(x)
def dequeue(self):
if not self.outbox:
while self.inbox: self.outbox.append(self.inbox.pop())
return self.outbox.pop()
```

Q6. Check if a binary tree is balanced.

● **HARD**

Answer:

For each node, recursively get heights of left and right subtrees. If they differ by more than 1, return -1 (unbalanced).

Hint: DFS returns height; -1 signals imbalanced

Code:

```
def is_balanced(root):
def height(node):
if not node: return 0
l = height(node.left)
r = height(node.right)
if l==-1 or r==-1 or abs(l-r)>1: return -1
return 1+max(l,r)
return height(root) != -1
```

Q7. Rotate a matrix 90 degrees clockwise in-place.

● **HARD**

Answer:

Step 1: transpose (swap $mat[i][j]$ and $mat[j][i]$). Step 2: reverse each row.

Hint: Transpose then reverse each row

Code:

```
def rotate(mat):
n = len(mat)
# Transpose
for i in range(n):
for j in range(i+1,n):
mat[i][j],mat[j][i] = mat[j][i],mat[i][j]
# Reverse each row
for row in mat:
row.reverse()
m=[[1,2,3],[4,5,6],[7,8,9]]; rotate(m); print(m)
```

Q8. Find if a path exists between two nodes in a directed graph (DFS).

● HARD

Answer:

Build adjacency list. DFS from source, mark visited. Return True when dest is reached.

Hint: DFS with visited set to avoid cycles

Code:

```
def has_path(edges, src, dst):
    graph = {}
    for u,v in edges:
        graph.setdefault(u, []).append(v)
    visited = set()
    def dfs(node):
        if node==dst: return True
        if node in visited: return False
        visited.add(node)
        return any(dfs(nb) for nb in graph.get(node, []))
    return dfs(src)
```

Q9. Implement Python's `__eq__`, `__lt__`, `__hash__` for a custom class.

● MED

Answer:

Define comparison operators on a frozen tuple of fields. `__hash__` must be consistent with `__eq__`.

Hint: Use tuple comparison for composite keys

Code:

```
class Point:
    def __init__(self,x,y): self.x,self.y=x,y
    def __eq__(self,o): return (self.x,self.y)==(o.x,o.y)
    def __lt__(self,o): return (self.x,self.y)<(o.x,o.y)
    def __hash__(self): return hash((self.x,self.y))
pts = {Point(1,2), Point(1,2)}
print(len(pts)) # 1 – deduped via __hash__/__eq__
```

Q10. Design a parking lot system using OOP.

● HARD

Answer:

Model each entity as a class. Use enums for vehicle type. ParkingLot coordinates floors; Floor manages slots.

Hint: Classes: ParkingLot, Floor, Slot, Vehicle

Code:

```
from enum import Enum

class VehicleType(Enum): BIKE=1; CAR=2; TRUCK=3

class Slot:
    def __init__(self, slot_id, vtype):
        self.id=slot_id; self.vtype=vtype; self.vehicle=None
```

```

@property
def is_free(self): return self.vehicle is None

class Floor:
def __init__(self, slots): self.slots=slots
def find_free(self, vtype):
return next((s for s in self.slots if s.is_free and s.vtype==vtype), None)

class ParkingLot:
def __init__(self, floors): self.floors=floors
def park(self, vehicle, vtype):
for f in self.floors:
slot = f.find_free(vtype)
if slot: slot.vehicle=vehicle; return slot.id
return None

```

Section C — SQL Coding Problems

Q1. Write a query using ROW_NUMBER() to rank employees by salary within each department.

● HARD

Answer:

ROW_NUMBER() is a window function that assigns a unique rank; DENSE_RANK() handles ties.

Code:

```

SELECT name, dept_id, salary,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dens_rnk
FROM Employees;

```

Q2. What is normalization? Explain 1NF, 2NF, 3NF with examples.

● HARD

Answer:

1NF: atomic values, no repeating groups (store phone numbers in a separate table). 2NF: 1NF + no partial dependency on composite key (order_id, product_id → price must not depend on product_id alone). 3NF: 2NF + no transitive dependency (employee_id → dept_id → dept_name: move dept_name to Departments table).

Code:

```

-- 3NF violation: dept_name depends on dept_id, not emp_id
-- Fix: separate Departments table
CREATE TABLE Departments (dept_id INT PK, dept_name VARCHAR);
CREATE TABLE Employees (emp_id INT PK, dept_id INT FK REFERENCES Departments);

```

Q3. Write a self-join query to find employees and their managers.

● HARD

Answer:

Self-join aliases the same table twice — one instance for employees, one for managers.

Code:

```
SELECT e.name AS employee, m.name AS manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.emp_id
ORDER BY manager;
```

Q4. What are CTEs (Common Table Expressions)? Write a recursive CTE.

● **HARD**

Answer:

A CTE (WITH clause) is a named temporary result set scoped to one query. Recursive CTEs have an anchor member (base case) and a recursive member joined with UNION ALL.

Code:

```
-- Recursive CTE: org hierarchy
WITH RECURSIVE Hierarchy AS (
SELECT id, name, manager_id, 1 AS lvl
FROM Employees WHERE manager_id IS NULL -- anchor
UNION ALL
SELECT e.id, e.name, e.manager_id, h.lvl+1
FROM Employees e
JOIN Hierarchy h ON e.manager_id = h.id -- recursive
)
SELECT * FROM Hierarchy ORDER BY lvl;
```

Q5. Find the cumulative sum of sales over time using window functions.

● **HARD**

Answer:

SUM() OVER with ORDER BY creates a running total. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW is the default frame.

Code:

```
SELECT order_date, amount,
SUM(amount) OVER (ORDER BY order_date
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM Sales;
```

Q6. What are stored procedures vs functions? When to use each?

● **HARD**

Answer:

Functions return a value, can be used in SELECT, cannot modify database state (in most DBs), no transaction control. Stored procedures can return multiple result sets, modify state, control transactions, cannot be used inline in queries. Use functions for computed columns; procedures for complex business logic.

Code:

```
-- Function (PostgreSQL)
CREATE FUNCTION get_tax(price NUMERIC) RETURNS NUMERIC AS $$
SELECT price * 0.18;
```



```
$$ LANGUAGE sql;
-- Use inline
SELECT product, get_tax(price) FROM Products;
```

Q7. Write a query to pivot rows into columns.

● **HARD**

Answer:

Use conditional aggregation (CASE WHEN or FILTER) to simulate PIVOT in databases that lack native PIVOT syntax.

Code:

```
SELECT dept_id,
SUM(CASE WHEN MONTH(sale_date)=1 THEN amount END) AS Jan,
SUM(CASE WHEN MONTH(sale_date)=2 THEN amount END) AS Feb,
SUM(CASE WHEN MONTH(sale_date)=3 THEN amount END) AS Mar
FROM Sales
GROUP BY dept_id;
```

Q8. What is an index scan vs index seek? When does a query not use an index?

● **HARD**

Answer:

Index seek: jumps directly to matching rows (highly selective). Index scan: scans all/most index pages (low selectivity). Index not used when: leading column not in WHERE, function applied to indexed column, type mismatch, OR conditions across columns, small table where full scan is cheaper.

Code:

```
-- Index NOT used (function on column)
SELECT * FROM Employees WHERE YEAR(hire_date)=2023;
-- Fix: range scan on raw column
SELECT * FROM Employees
WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Q9. Explain transactions and isolation levels.

● **HARD**

Answer:

Read Uncommitted: dirty reads allowed. Read Committed (default PG): sees only committed data; non-repeatable reads possible. Repeatable Read: same row reads consistent within txn; phantom reads possible. Serializable: full isolation; no phantoms. Higher isolation = more locking = less concurrency.

Code:

```
-- PostgreSQL
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id=1;
-- Another txn commits a change here
SELECT balance FROM accounts WHERE id=1; -- same value guaranteed
COMMIT;
```

Q10. Find records that changed between two snapshots of a table.

● **HARD**

Answer:

EXCEPT (or MINUS) finds rows in snapshot A not in snapshot B. Join on primary key to show old vs new values.

Code:

```
-- Rows in old snapshot not in new (changed or deleted)
SELECT * FROM employees_snapshot_old
EXCEPT
SELECT * FROM employees_snapshot_new;

-- Show old vs new side by side
SELECT o.id, o.salary AS old_sal, n.salary AS new_sal
FROM employees_snapshot_old o
JOIN employees_snapshot_new n USING (id)
WHERE o.salary <> n.salary;
```

Section D — DSA Problems

Q1. Find the intersection point of two linked lists.

● **HARD**

Answer:

Calculate lengths. Advance pointer of longer list by the difference. Then move both together — they meet at the intersection.

Hint: Equalise lengths by advancing the longer list pointer

Code:

```
def get_intersection(h1, h2):
    l1=l2=0; a,b=h1,h2
    while a: l1+=1; a=a.next
    while b: l2+=1; b=b.next
    a,b=h1,h2
    if l1>l2:
        for _ in range(l1-l2): a=a.next
    else:
        for _ in range(l2-l1): b=b.next
    while a is not b: a=a.next; b=b.next
    return a
```

Q2. Remove the nth node from the end of a linked list in one pass.

● **HARD**

Answer:

Advance fast pointer n+1 steps. Move both until fast is None. Slow is now at node before the target.

Hint: Two pointers n+1 apart

Code:

```
def remove_nth(head, n):
    dummy = Node(0); dummy.next=head
    fast=slow=dummy
    for _ in range(n+1): fast=fast.next
    while fast: fast=fast.next; slow=slow.next
    slow.next=slow.next.next
    return dummy.next
```

Q3. Sort a linked list using merge sort.

● **HARD**

Answer:

Find mid with slow/fast. Split. Recursively sort halves. Merge sorted halves.

Hint: Split at mid, sort halves, merge

Code:

```
def sort_list(head):
    if not head or not head.next: return head
    slow, fast=head, head.next
    while fast and fast.next: slow=slow.next; fast=fast.next.next
    mid=slow.next; slow.next=None
    l=sort_list(head); r=sort_list(mid)
    dummy=Node(0); cur=dummy
    while l and r:
        if l.val<r.val: cur.next=l; l=l.next
        else: cur.next=r; r=r.next
    cur=cur.next
    cur.next=l or r
    return dummy.next
```

Day 3 — OS & Computer Networks

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 — OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 — OS & Computer Networks
# Sample implementation pattern
```

```
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example

```


Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 3 – OS & Computer Networks
# Sample implementation pattern
def solution_3(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_3([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 3 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 3 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 3 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 3 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 3 SQL – OS & Computer Networks
WITH base AS (

```



```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 3 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 3 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 3 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 3 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 3 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 3 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 4 — System Design Fundamentals

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 4 – System Design Fundamentals
# Sample implementation pattern
def solution_4(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_4([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 4 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 4 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 4 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 4 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 4 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 4 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 4 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 4 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 4 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL -- System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL -- System Design Fundamentals
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 4 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 4 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 4 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 4 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 5 — Web Dev: Django & FastAPI

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 5 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_5(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_5([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```



```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 5 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 5 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 5 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 5 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```



```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 5 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 5 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 5 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 5 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 5 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 5 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 6 — Machine Learning Foundations

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 6 – Machine Learning Foundations
# Sample implementation pattern
def solution_6(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_6([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 6 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 6 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 6 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 6 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 6 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 6 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL -- Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 6 SQL -- Machine Learning Foundations
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 6 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 6 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 6 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 6 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 7 — Deep Learning & Neural Networks

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 7 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_7(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_7([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 7 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 7 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 7 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 7 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 7 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 7 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 7 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 7 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 7 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 8 — Data Engineering & ETL

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 8 – Data Engineering & ETL
# Sample implementation pattern
def solution_8(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_8([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 8 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 8 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 8 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 8 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 8 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 8 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 8 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 9 — Cloud & DevOps

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 9 – Cloud & DevOps
# Sample implementation pattern
def solution_9(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_9([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```

```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 9 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 9 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 9 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 9 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 9 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 9 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 9 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 9 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 9 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 10 — Software Testing & QA

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
```



```
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 10 – Software Testing & QA
# Sample implementation pattern
def solution_10(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_10([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 10 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 10 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 10 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 10 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 10 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 10 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 10 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 10 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 10 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```



```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 10 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 10 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 11 — NLP & Large Language Models

NLP + LLMs · DSA: Advanced Graphs

Topics: NLP · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 11 – NLP & Large Language Models
# Sample implementation pattern
def solution_11(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_11([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic NLP & Large Language Models coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 11 Coding Problem 1 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 11 Coding Problem 2 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```



```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 11 Coding Problem 3 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world NLP & Large Language Models system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 11 Coding Problem 4 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 5 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 6 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 7 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 8 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 9 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 11 Coding Problem 10 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale NLP & Large Language Models system.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse NLP & Large Language Models metrics.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in NLP & Large Language Models data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for NLP & Large Language Models.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 11 SQL – NLP & Large Language Models

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for NLP & Large Language Models events.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a NLP & Large Language Models platform.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for NLP & Large Language Models data.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
```



```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a NLP & Large Language Models pipeline table.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent NLP & Large Language Models ETL.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for NLP & Large Language Models users.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 11 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard NLP & Large Language Models DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 11 DSA 1 – NLP & Large Language Models
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient NLP & Large Language Models algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 11 DSA 2 – NLP & Large Language Models
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the NLP & Large Language Models variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 11 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 12 — Advanced Python & Concurrency

Async + Internals · DSA: Backtracking

Topics: Python · Concurrency

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 12 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_12(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_12([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Advanced Python & Concurrency coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 12 Coding Problem 1 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 12 Coding Problem 2 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 12 Coding Problem 3 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Advanced Python & Concurrency system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 12 Coding Problem 4 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 5 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 6 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 7 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 8 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 9 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 12 Coding Problem 10 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Advanced Python & Concurrency system.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Advanced Python & Concurrency metrics.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Advanced Python & Concurrency data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Advanced Python & Concurrency.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Advanced Python & Concurrency events.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Advanced Python & Concurrency platform.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Advanced Python & Concurrency data.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Advanced Python & Concurrency pipeline table.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Advanced Python & Concurrency ETL.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Advanced Python & Concurrency users.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 12 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Advanced Python & Concurrency DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 12 DSA 1 – Advanced Python & Concurrency
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Advanced Python & Concurrency algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 12 DSA 2 – Advanced Python & Concurrency
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Advanced Python & Concurrency variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 12 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 13 — OS & Computer Networks — Intermediate

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
```



```
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 13 – OS & Computer Networks
# Sample implementation pattern
def solution_13(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_13([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 13 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 13 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 13 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 13 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 13 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 13 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 13 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 14 — System Design Fundamentals — Intermediate

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 14 – System Design Fundamentals
# Sample implementation pattern
def solution_14(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```



```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_14([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 14 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 14 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 14 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 14 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 14 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 14 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 14 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 14 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 14 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 14 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 14 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 14 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 14 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 15 — Web Dev: Django & FastAPI — Intermediate

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 15 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_15(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_15([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 15 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 15 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 15 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 15 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 15 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 15 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 15 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 16 — Machine Learning Foundations — Intermediate

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 16 – Machine Learning Foundations
# Sample implementation pattern
def solution_16(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_16([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 16 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 16 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 16 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 16 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```



```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 16 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 16 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
SELECT
```



```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 16 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 16 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 16 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 16 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 17 — Deep Learning & Neural Networks — Intermediate

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 17 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_17(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_17([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 17 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 17 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 17 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 17 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 17 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks

```



```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 17 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 17 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 17 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 17 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 18 — Data Engineering & ETL — Intermediate

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 18 – Data Engineering & ETL
# Sample implementation pattern
def solution_18(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_18([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 18 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 18 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 18 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 18 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 18 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 18 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 18 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 19 — Cloud & DevOps — Intermediate

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 19 – Cloud & DevOps
# Sample implementation pattern
def solution_19(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_19([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```



```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 19 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 19 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 19 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 19 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```


Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● HARD

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● HARD

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 19 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 19 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 19 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 19 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 19 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 20 — Software Testing & QA — Intermediate

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
```

```
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 20 – Software Testing & QA
# Sample implementation pattern
def solution_20(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_20([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 20 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 20 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 20 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 20 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 20 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```



```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 20 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 20 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 20 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 20 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```

```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 20 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 20 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 21 — NLP & Large Language Models — Intermediate

NLP + LLMs · DSA: Advanced Graphs

Topics: NLP · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```



```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 21 – NLP & Large Language Models
# Sample implementation pattern
def solution_21(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_21([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic NLP & Large Language Models coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 21 Coding Problem 1 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 21 Coding Problem 2 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 21 Coding Problem 3 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world NLP & Large Language Models system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 21 Coding Problem 4 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 5 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 6 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 7 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 8 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 9 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 21 Coding Problem 10 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale NLP & Large Language Models system.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse NLP & Large Language Models metrics.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in NLP & Large Language Models data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for NLP & Large Language Models.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 21 SQL – NLP & Large Language Models

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for NLP & Large Language Models events.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a NLP & Large Language Models platform.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for NLP & Large Language Models data.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a NLP & Large Language Models pipeline table.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent NLP & Large Language Models ETL.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for NLP & Large Language Models users.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 21 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard NLP & Large Language Models DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 21 DSA 1 – NLP & Large Language Models
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient NLP & Large Language Models algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 21 DSA 2 – NLP & Large Language Models
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the NLP & Large Language Models variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 21 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 22 — Advanced Python & Concurrency — Intermediate

Async + Internals · DSA: Backtracking

Topics: Python · Concurrency

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 22 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_22(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_22([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Advanced Python & Concurrency coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 22 Coding Problem 1 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 22 Coding Problem 2 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 22 Coding Problem 3 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Advanced Python & Concurrency system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 22 Coding Problem 4 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```



```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 5 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 6 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 7 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 8 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 9 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 22 Coding Problem 10 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Advanced Python & Concurrency system.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Advanced Python & Concurrency metrics.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Advanced Python & Concurrency data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Advanced Python & Concurrency.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Advanced Python & Concurrency events.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Advanced Python & Concurrency platform.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Advanced Python & Concurrency data.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Advanced Python & Concurrency pipeline table.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Advanced Python & Concurrency ETL.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency

```



```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Advanced Python & Concurrency users.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 22 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Advanced Python & Concurrency DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 22 DSA 1 – Advanced Python & Concurrency
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Advanced Python & Concurrency algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 22 DSA 2 – Advanced Python & Concurrency
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Advanced Python & Concurrency variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 22 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 23 — OS & Computer Networks — Intermediate

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
```

```
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example

```


Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 23 – OS & Computer Networks
# Sample implementation pattern
def solution_23(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_23([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 23 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 23 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 23 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 23 SQL – OS & Computer Networks
WITH base AS (

```



```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 23 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 23 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 23 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 23 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 24 — System Design Fundamentals — Intermediate

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 24 – System Design Fundamentals
# Sample implementation pattern
def solution_24(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_24([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 24 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 24 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 24 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 24 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 24 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 24 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 24 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 24 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 24 SQL – System Design Fundamentals
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 24 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 24 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 24 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 24 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 25 — Web Dev: Django & FastAPI — Advanced

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 25 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_25(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_25([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```



```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 25 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 25 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 25 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```



```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 25 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 25 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 25 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 25 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 25 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 25 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 26 — Machine Learning Foundations — Advanced

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 26 – Machine Learning Foundations
# Sample implementation pattern
def solution_26(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_26([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 26 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 26 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 26 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 26 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 26 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 26 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 26 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 26 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 26 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 26 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 27 — Deep Learning & Neural Networks — Advanced

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 27 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_27(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_27([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 27 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 27 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 27 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 27 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 27 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 27 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 27 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 27 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 27 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 28 — Data Engineering & ETL — Advanced

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 28 – Data Engineering & ETL
# Sample implementation pattern
def solution_28(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_28([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 28 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 28 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 28 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,

```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 28 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 28 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 28 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 28 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 29 — Cloud & DevOps — Advanced

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 29 – Cloud & DevOps
# Sample implementation pattern
def solution_29(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result
if __name__ == '__main__':
    print(solution_29([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```

```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 29 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 29 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 29 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 29 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 29 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 29 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 29 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 29 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 29 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 30 — Software Testing & QA — Advanced

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
```



```
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 30 – Software Testing & QA
# Sample implementation pattern
def solution_30(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_30([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 30 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 30 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 30 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 30 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 30 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 30 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```



```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 30 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 30 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 31 — NLP & Large Language Models — Advanced

NLP + LLMs · DSA: Advanced Graphs

Topics: NLP · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 31 – NLP & Large Language Models
# Sample implementation pattern
def solution_31(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_31([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic NLP & Large Language Models coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 31 Coding Problem 1 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 31 Coding Problem 2 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```



```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 31 Coding Problem 3 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world NLP & Large Language Models system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 31 Coding Problem 4 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 5 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 6 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 7 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 8 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 9 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 31 Coding Problem 10 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale NLP & Large Language Models system.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse NLP & Large Language Models metrics.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in NLP & Large Language Models data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for NLP & Large Language Models.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 31 SQL – NLP & Large Language Models

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for NLP & Large Language Models events.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a NLP & Large Language Models platform.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for NLP & Large Language Models data.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
```



```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a NLP & Large Language Models pipeline table.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent NLP & Large Language Models ETL.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for NLP & Large Language Models users.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 31 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard NLP & Large Language Models DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 31 DSA 1 – NLP & Large Language Models
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient NLP & Large Language Models algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 31 DSA 2 – NLP & Large Language Models
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the NLP & Large Language Models variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 31 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 32 — Advanced Python & Concurrency — Advanced

Async + Internals · DSA: Backtracking

Topics: Python · Concurrency

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 32 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_32(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_32([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Advanced Python & Concurrency coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 32 Coding Problem 1 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 32 Coding Problem 2 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 32 Coding Problem 3 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Advanced Python & Concurrency system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 32 Coding Problem 4 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 5 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 6 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 7 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 8 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 9 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 32 Coding Problem 10 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Advanced Python & Concurrency system.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Advanced Python & Concurrency metrics.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Advanced Python & Concurrency data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Advanced Python & Concurrency.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Advanced Python & Concurrency events.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Advanced Python & Concurrency platform.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Advanced Python & Concurrency data.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Advanced Python & Concurrency pipeline table.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Advanced Python & Concurrency ETL.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Advanced Python & Concurrency users.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 32 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Advanced Python & Concurrency DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 32 DSA 1 – Advanced Python & Concurrency
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Advanced Python & Concurrency algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 32 DSA 2 – Advanced Python & Concurrency
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Advanced Python & Concurrency variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 32 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 33 — OS & Computer Networks — Advanced

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
```



```
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 33 – OS & Computer Networks
# Sample implementation pattern
def solution_33(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result
if __name__ == '__main__':
    print(solution_33([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 33 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 33 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 33 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (

```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 33 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 33 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,

```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 33 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 33 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 33 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 33 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 34 — System Design Fundamentals — Advanced

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 34 – System Design Fundamentals
# Sample implementation pattern
def solution_34(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```



```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_34([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 34 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 34 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 34 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 34 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 34 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 34 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 34 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 34 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 34 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 34 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 34 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 34 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 34 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 35 — Web Dev: Django & FastAPI — Advanced

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 35 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_35(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result
if __name__ == '__main__':
    print(solution_35([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 35 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 35 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 35 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 35 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 35 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT

```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 35 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 35 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 35 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 35 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 36 — Machine Learning Foundations — Advanced

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 36 – Machine Learning Foundations
# Sample implementation pattern
def solution_36(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_36([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 36 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 36 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 36 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 36 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```



```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 36 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 36 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
SELECT
```



```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 36 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 36 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 36 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 36 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 37 — Deep Learning & Neural Networks — Advanced

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```


Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 37 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_37(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_37([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 37 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 37 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 37 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 37 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 37 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks

```



```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 37 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 37 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 37 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 37 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 38 — Data Engineering & ETL — Advanced

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```


Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 38 – Data Engineering & ETL
# Sample implementation pattern
def solution_38(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_38([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 38 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 38 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 38 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 38 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 38 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 38 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 38 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 39 — Cloud & DevOps — Advanced

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 39 – Cloud & DevOps
# Sample implementation pattern
def solution_39(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_39([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```



```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 39 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 39 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 39 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 39 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```


Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 39 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 39 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 39 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 39 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 39 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 40 — Software Testing & QA — Advanced

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 — Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 — Software Testing & QA
# Sample implementation pattern
```

```
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example

```


Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 40 – Software Testing & QA
# Sample implementation pattern
def solution_40(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_40([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 40 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 40 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 40 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```



```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 40 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 40 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 40 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```

```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 40 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 40 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 41 — NLP & Large Language Models — Advanced

NLP + LLMs · DSA: Advanced Graphs

Topics: NLP · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```



```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 41 – NLP & Large Language Models
# Sample implementation pattern
def solution_41(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_41([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic NLP & Large Language Models coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 41 Coding Problem 1 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 41 Coding Problem 2 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 41 Coding Problem 3 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world NLP & Large Language Models system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 41 Coding Problem 4 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 5 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 6 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 7 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 8 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 9 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 41 Coding Problem 10 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale NLP & Large Language Models system.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse NLP & Large Language Models metrics.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in NLP & Large Language Models data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for NLP & Large Language Models.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 41 SQL – NLP & Large Language Models

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for NLP & Large Language Models events.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a NLP & Large Language Models platform.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for NLP & Large Language Models data.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a NLP & Large Language Models pipeline table.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent NLP & Large Language Models ETL.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for NLP & Large Language Models users.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 41 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard NLP & Large Language Models DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 41 DSA 1 – NLP & Large Language Models
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient NLP & Large Language Models algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 41 DSA 2 – NLP & Large Language Models
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the NLP & Large Language Models variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 41 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 42 — Advanced Python & Concurrency — Advanced

Async + Internals · DSA: Backtracking

Topics: Python · Concurrency

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 42 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_42(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_42([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Advanced Python & Concurrency coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 42 Coding Problem 1 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 42 Coding Problem 2 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 42 Coding Problem 3 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Advanced Python & Concurrency system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 42 Coding Problem 4 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```



```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 5 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 6 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 7 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 8 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 9 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 42 Coding Problem 10 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Advanced Python & Concurrency system.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Advanced Python & Concurrency metrics.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Advanced Python & Concurrency data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Advanced Python & Concurrency.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Advanced Python & Concurrency events.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Advanced Python & Concurrency platform.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Advanced Python & Concurrency data.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Advanced Python & Concurrency pipeline table.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Advanced Python & Concurrency ETL.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency

```



```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Advanced Python & Concurrency users.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 42 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Advanced Python & Concurrency DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 42 DSA 1 – Advanced Python & Concurrency
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Advanced Python & Concurrency algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 42 DSA 2 – Advanced Python & Concurrency
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Advanced Python & Concurrency variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 42 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 43 — OS & Computer Networks — Advanced

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
```

```
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example

```


Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 43 – OS & Computer Networks
# Sample implementation pattern
def solution_43(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_43([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 43 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 43 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 43 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 43 SQL – OS & Computer Networks
WITH base AS (

```



```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 43 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 43 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 43 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 43 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 44 — System Design Fundamentals — Advanced

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 44 – System Design Fundamentals
# Sample implementation pattern
def solution_44(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_44([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 44 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 44 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 44 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 44 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 44 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 44 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 44 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 44 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 44 SQL – System Design Fundamentals
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 44 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 44 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 44 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 44 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 45 — Web Dev: Django & FastAPI — Advanced

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 45 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_45(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_45([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```



```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 45 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 45 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 45 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```



```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 45 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 45 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 45 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 45 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 45 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 46 — Machine Learning Foundations — Advanced

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 46 – Machine Learning Foundations
# Sample implementation pattern
def solution_46(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_46([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 46 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 46 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 46 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 46 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 46 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 46 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 46 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 46 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 46 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 46 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 47 — Deep Learning & Neural Networks — Advanced

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 — Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 47 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_47(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_47([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 47 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 47 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 47 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 47 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 47 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 47 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 47 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 47 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 47 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 48 — Data Engineering & ETL — Advanced

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 48 – Data Engineering & ETL
# Sample implementation pattern
def solution_48(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_48([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 48 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 48 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 48 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 48 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 48 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 48 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 48 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 49 — Cloud & DevOps — Advanced

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 49 – Cloud & DevOps
# Sample implementation pattern
def solution_49(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_49([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```

```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 49 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 49 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 49 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 49 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 49 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 49 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 49 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 49 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 49 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 50 — Software Testing & QA — Advanced

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 — Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 — Software Testing & QA
# Sample implementation pattern
```



```
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 50 – Software Testing & QA
# Sample implementation pattern
def solution_50(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_50([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 50 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 50 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 50 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 50 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 50 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 50 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 50 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 50 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 50 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```



```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 50 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 50 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 51 — NLP & Large Language Models — Advanced

NLP + LLMs · DSA: Advanced Graphs

Topics: NLP · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of NLP & Large Language Models: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 51 – NLP & Large Language Models
# Sample implementation pattern
def solution_51(data):
    """
    Production pattern for NLP & Large Language Models.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_51([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic NLP & Large Language Models coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 51 Coding Problem 1 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 51 Coding Problem 2 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```



```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 51 Coding Problem 3 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world NLP & Large Language Models system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```

# Day 51 Coding Problem 4 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 5 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 6 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 7 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 8 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 9 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for NLP & Large Language Models problems at this difficulty.

Code:

```
# Day 51 Coding Problem 10 – NLP & Large Language Models
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale NLP & Large Language Models system.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse NLP & Large Language Models metrics.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in NLP & Large Language Models data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for NLP & Large Language Models.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 51 SQL – NLP & Large Language Models

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for NLP & Large Language Models events.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a NLP & Large Language Models platform.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for NLP & Large Language Models data.

● **MED**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'NLP & La'
  GROUP BY 1, 2
),
```



```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a NLP & Large Language Models pipeline table.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent NLP & Large Language Models ETL.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for NLP & Large Language Models users.

● **HARD**

Answer:

For NLP & Large Language Models SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 51 SQL – NLP & Large Language Models
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'NLP & La'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard NLP & Large Language Models DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 51 DSA 1 – NLP & Large Language Models
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient NLP & Large Language Models algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 51 DSA 2 – NLP & Large Language Models
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the NLP & Large Language Models variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 51 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 52 — Advanced Python & Concurrency — Advanced

Async + Internals · DSA: Backtracking

Topics: Python · Concurrency

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Advanced Python & Concurrency: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 52 – Advanced Python & Concurrency
# Sample implementation pattern
def solution_52(data):
    """
    Production pattern for Advanced Python & Concurrency.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_52([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Advanced Python & Concurrency coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 52 Coding Problem 1 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 52 Coding Problem 2 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 52 Coding Problem 3 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Advanced Python & Concurrency system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```

# Day 52 Coding Problem 4 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 5 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 6 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 7 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 8 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 9 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Advanced Python & Concurrency problems at this difficulty.

Code:

```
# Day 52 Coding Problem 10 – Advanced Python & Concurrency
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Advanced Python & Concurrency system.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Advanced Python & Concurrency metrics.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```



```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Advanced Python & Concurrency data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Advanced Python & Concurrency.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Advanced Python & Concurrency events.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Advanced Python & Concurrency platform.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Advanced Python & Concurrency data.

● **MED**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Advanced'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Advanced Python & Concurrency pipeline table.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Advanced Python & Concurrency ETL.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Advanced Python & Concurrency users.

● **HARD**

Answer:

For Advanced Python & Concurrency SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 52 SQL – Advanced Python & Concurrency
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Advanced'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Advanced Python & Concurrency DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 52 DSA 1 – Advanced Python & Concurrency
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Advanced Python & Concurrency algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 52 DSA 2 – Advanced Python & Concurrency
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Advanced Python & Concurrency variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 52 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 53 — OS & Computer Networks — Advanced

OS + CN · DSA: Stacks & Queues

Topics: OS · Networks

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
```



```
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
```

```

"""
Production pattern for OS & Computer Networks.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of OS & Computer Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 53 – OS & Computer Networks
# Sample implementation pattern
def solution_53(data):
    """
    Production pattern for OS & Computer Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_53([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic OS & Computer Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 1 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 2 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 53 Coding Problem 3 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world OS & Computer Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```

# Day 53 Coding Problem 4 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 5 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 6 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 7 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 8 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 9 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for OS & Computer Networks problems at this difficulty.

Code:

```
# Day 53 Coding Problem 10 – OS & Computer Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale OS & Computer Networks system.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse OS & Computer Networks metrics.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
```

```

ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in OS & Computer Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for OS & Computer Networks.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 53 SQL – OS & Computer Networks
WITH base AS (

```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for OS & Computer Networks events.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a OS & Computer Networks platform.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for OS & Computer Networks data.

● **MED**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'OS & Com'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a OS & Computer Networks pipeline table.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent OS & Computer Networks ETL.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for OS & Computer Networks users.

● **HARD**

Answer:

For OS & Computer Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 53 SQL – OS & Computer Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'OS & Com'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q1. Solve a hard OS & Computer Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 53 DSA 1 – OS & Computer Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient OS & Computer Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 53 DSA 2 – OS & Computer Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the OS & Computer Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 53 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 54 — System Design Fundamentals — Advanced

System Design + APIs · DSA: Trees

Topics: System Design · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● HARD

Answer:

In the context of System Design Fundamentals: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 54 – System Design Fundamentals
# Sample implementation pattern
def solution_54(data):
    """
    Production pattern for System Design Fundamentals.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```



```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_54([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic System Design Fundamentals coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 54 Coding Problem 1 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 54 Coding Problem 2 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 54 Coding Problem 3 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world System Design Fundamentals system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```

# Day 54 Coding Problem 4 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 5 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 6 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 7 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 8 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 9 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for System Design Fundamentals problems at this difficulty.

Code:

```
# Day 54 Coding Problem 10 – System Design Fundamentals
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale System Design Fundamentals system.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse System Design Fundamentals metrics.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in System Design Fundamentals data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 54 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for System Design Fundamentals.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 54 SQL – System Design Fundamentals

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for System Design Fundamentals events.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 54 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```


Q6. Design a schema supporting multi-tenant access control for a System Design Fundamentals platform.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for System Design Fundamentals data.

● **MED**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'System D'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a System Design Fundamentals pipeline table.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent System Design Fundamentals ETL.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 54 SQL – System Design Fundamentals
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for System Design Fundamentals users.

● **HARD**

Answer:

For System Design Fundamentals SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 54 SQL – System Design Fundamentals
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'System D'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard System Design Fundamentals DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 54 DSA 1 – System Design Fundamentals
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient System Design Fundamentals algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 54 DSA 2 – System Design Fundamentals
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the System Design Fundamentals variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 54 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 55 — Web Dev: Django & FastAPI — Advanced

Python Web Frameworks · DSA: Heaps

Topics: Web Dev · Backend

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 — Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 — Web Dev: Django & FastAPI
# Sample implementation pattern
```

```
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time:  $O(n \log n)$  | Space:  $O(n)$ 
    """
    # Step 1: Validate input
    if not data:
        return None
```



```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
```

```

"""
Production pattern for Web Dev: Django & FastAPI.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Web Dev: Django & FastAPI: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 55 – Web Dev: Django & FastAPI
# Sample implementation pattern
def solution_55(data):
    """
    Production pattern for Web Dev: Django & FastAPI.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_55([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Web Dev: Django & FastAPI coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 1 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 2 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 55 Coding Problem 3 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Web Dev: Django & FastAPI system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```

# Day 55 Coding Problem 4 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 5 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 6 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 7 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 8 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 9 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Web Dev: Django & FastAPI problems at this difficulty.

Code:

```
# Day 55 Coding Problem 10 – Web Dev: Django & FastAPI
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```


Q1. Design and query a database schema for a large-scale Web Dev: Django & FastAPI system.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Web Dev: Django & FastAPI metrics.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q3. Find anomalies in Web Dev: Django & FastAPI data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q4. Implement a reporting query with CTEs and recursive elements for Web Dev: Django & FastAPI.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
```

```

SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Web Dev: Django & FastAPI events.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Web Dev: Django & FastAPI platform.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Web Dev: Django & FastAPI data.

● **MED**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Web Dev:'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Web Dev: Django & FastAPI pipeline table.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Web Dev: Django & FastAPI ETL.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 55 SQL -- Web Dev: Django & FastAPI
WITH base AS (
SELECT
```

```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Web Dev: Django & FastAPI users.

● **HARD**

Answer:

For Web Dev: Django & FastAPI SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 55 SQL – Web Dev: Django & FastAPI
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Web Dev:'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Web Dev: Django & FastAPI DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 55 DSA 1 – Web Dev: Django & FastAPI
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Web Dev: Django & FastAPI algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 55 DSA 2 – Web Dev: Django & FastAPI
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Web Dev: Django & FastAPI variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 55 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 56 — Machine Learning Foundations — Advanced

ML Core Concepts · DSA: Graphs

Topics: ML · Data Science

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Machine Learning Foundations: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 56 – Machine Learning Foundations
# Sample implementation pattern
def solution_56(data):
    """
    Production pattern for Machine Learning Foundations.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_56([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Machine Learning Foundations coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 56 Coding Problem 1 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 56 Coding Problem 2 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 56 Coding Problem 3 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Machine Learning Foundations system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```

# Day 56 Coding Problem 4 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```



```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 5 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 6 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 7 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 8 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 9 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Machine Learning Foundations problems at this difficulty.

Code:

```
# Day 56 Coding Problem 10 – Machine Learning Foundations
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Machine Learning Foundations system.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Machine Learning Foundations metrics.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Machine Learning Foundations data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Machine Learning Foundations.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 56 SQL – Machine Learning Foundations

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Machine Learning Foundations events.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Machine Learning Foundations platform.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Machine Learning Foundations data.

● **MED**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Machine '
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Machine Learning Foundations pipeline table.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Machine Learning Foundations ETL.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
SELECT
```



```

user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Machine Learning Foundations users.

● **HARD**

Answer:

For Machine Learning Foundations SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 56 SQL – Machine Learning Foundations
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Machine '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Machine Learning Foundations DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 56 DSA 1 – Machine Learning Foundations
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Machine Learning Foundations algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 56 DSA 2 – Machine Learning Foundations
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Machine Learning Foundations variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 56 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```

```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 57 — Deep Learning & Neural Networks — Advanced

DL Fundamentals · DSA: Dynamic Programming I

Topics: DL · AI

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example

```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example

```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Deep Learning & Neural Networks: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 57 – Deep Learning & Neural Networks
# Sample implementation pattern
def solution_57(data):
    """
    Production pattern for Deep Learning & Neural Networks.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
```

```

if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_57([3,1,4,1,5,9])) # example

```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Deep Learning & Neural Networks coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 57 Coding Problem 1 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 57 Coding Problem 2 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)

```

```

if n == 0: return 0
dp = [0] * (n + 1)
dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 57 Coding Problem 3 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Deep Learning & Neural Networks system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```

# Day 57 Coding Problem 4 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):

```

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 5 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 6 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 7 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 8 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 9 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Deep Learning & Neural Networks problems at this difficulty.

Code:

```
# Day 57 Coding Problem 10 – Deep Learning & Neural Networks
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Deep Learning & Neural Networks system.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Deep Learning & Neural Networks metrics.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Deep Learning & Neural Networks data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Deep Learning & Neural Networks.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks

```



```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Deep Learning & Neural Networks events.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Deep Learning & Neural Networks platform.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Deep Learning & Neural Networks data.

● **MED**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Deep Lea'
  GROUP BY 1, 2
```

```

),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q8. Write a query to detect data quality issues in a Deep Learning & Neural Networks pipeline table.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Deep Learning & Neural Networks ETL.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks

```

```

WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Deep Learning & Neural Networks users.

● **HARD**

Answer:

For Deep Learning & Neural Networks SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 57 SQL – Deep Learning & Neural Networks
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Deep Lea'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Deep Learning & Neural Networks DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 57 DSA 1 – Deep Learning & Neural Networks
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Deep Learning & Neural Networks algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 57 DSA 2 – Deep Learning & Neural Networks
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Deep Learning & Neural Networks variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 57 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
```

```
for c in choices:
    if is_valid(c, path):
        path.append(c)
        solve(remaining(choices, c), path, results)
        path.pop()
```

Day 58 — Data Engineering & ETL — Advanced

Data Pipelines · DSA: DP II

Topics: Data Engineering · Python

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
```

```
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```


Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
```

```

"""
Production pattern for Data Engineering & ETL.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Data Engineering & ETL: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 58 – Data Engineering & ETL
# Sample implementation pattern
def solution_58(data):
    """
    Production pattern for Data Engineering & ETL.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_58([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Data Engineering & ETL coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 1 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 2 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 58 Coding Problem 3 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Data Engineering & ETL system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```

# Day 58 Coding Problem 4 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 5 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 6 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 7 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 8 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 9 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Data Engineering & ETL problems at this difficulty.

Code:

```
# Day 58 Coding Problem 10 – Data Engineering & ETL
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Data Engineering & ETL system.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Data Engineering & ETL metrics.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Data Engineering & ETL data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Data Engineering & ETL.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,

```

```

DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Data Engineering & ETL events.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Data Engineering & ETL platform.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Data Engineering & ETL data.

● **MED**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Data Eng'
  GROUP BY 1, 2
),
```

```
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Data Engineering & ETL pipeline table.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Data Engineering & ETL ETL.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
```

```

COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Data Engineering & ETL users.

● **HARD**

Answer:

For Data Engineering & ETL SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 58 SQL – Data Engineering & ETL
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Data Eng'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q1. Solve a hard Data Engineering & ETL DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```
# Day 58 DSA 1 – Data Engineering & ETL
@functools.lru_cache(maxsize=None)
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Data Engineering & ETL algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 58 DSA 2 – Data Engineering & ETL
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Data Engineering & ETL variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 58 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
```



```
if is_valid(c, path):  
    path.append(c)  
    solve(remaining(choices, c), path, results)  
    path.pop()
```

Day 59 — Cloud & DevOps — Advanced

AWS/GCP + Docker/K8s · DSA: Greedy

Topics: Cloud · DevOps

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
```

```
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
```

```

"""
Production pattern for Cloud & DevOps.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Cloud & DevOps: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 59 – Cloud & DevOps
# Sample implementation pattern
def solution_59(data):
    """
    Production pattern for Cloud & DevOps.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_59([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Cloud & DevOps coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 1 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 2 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
```



```
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 3 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q4. Design a class hierarchy using OOP principles for a real-world Cloud & DevOps system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 4 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 5 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 6 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 7 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 8 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 9 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Cloud & DevOps problems at this difficulty.

Code:

```
# Day 59 Coding Problem 10 – Cloud & DevOps
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Cloud & DevOps system.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Cloud & DevOps metrics.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Cloud & DevOps data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 59 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Cloud & DevOps.

● **MED**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 59 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```

```

SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Cloud & DevOps events.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 59 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Cloud & DevOps platform.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Cloud & DevOps data.

● MED

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```


Q8. Write a query to detect data quality issues in a Cloud & DevOps pipeline table.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Cloud & DevOps ETL.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 59 SQL – Cloud & DevOps
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Cloud & '
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Cloud & DevOps users.

● **HARD**

Answer:

For Cloud & DevOps SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 59 SQL – Cloud & DevOps
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Cloud & '
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Cloud & DevOps DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 59 DSA 1 – Cloud & DevOps
@functools.lru_cache(maxsize=None)
def dp(i, j):

```

```
if i == 0 or j == 0: return 0
if s1[i-1] == s2[j-1]:
    return 1 + dp(i-1, j-1)
return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Cloud & DevOps algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 59 DSA 2 – Cloud & DevOps
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Cloud & DevOps variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 59 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
    return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```

Day 60 — Software Testing & QA — Advanced

Testing Strategies · DSA: Sorting

Topics: Testing · QA

Section A — Interview Questions

Q1. Explain the key concepts and internals of this topic with a real production example.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 — Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q2. What are the most common pitfalls engineers face with this topic in interviews?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 — Software Testing & QA
# Sample implementation pattern
```

```
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q3. Compare the top 3 approaches to solving problems in this domain with trade-offs.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q4. Walk through a system design scenario applying this topic at scale (10M users).

● MED

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q5. What monitoring, observability, and debugging techniques apply to this domain?

● HARD

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q6. Explain the underlying data structures or algorithms powering this technology.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q7. How do you handle failure modes and edge cases in this topic area?

● **MED**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
```

```

"""
Production pattern for Software Testing & QA.
Time: O(n log n) | Space: O(n)
"""

# Step 1: Validate input
if not data:
    return None
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example

```

Q8. Describe a recent project where you applied this concept and what you learned.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```

# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """

    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example

```

Q9. What are the key differences between junior and senior understanding of this topic?

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
    # Step 2: Core algorithm
    result = sorted(data, key=lambda x: x) # adapt per problem
    # Step 3: Post-process
    return result
if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Q10. List the top 5 resources, tools, and open-source projects for mastering this area.

● **HARD**

Answer:

In the context of Software Testing & QA: This is a depth question testing your understanding of core internals, trade-offs, and production readiness. A strong answer covers: (1) fundamental concepts with precise terminology, (2) at least one concrete real-world example or system, (3) known pitfalls and how to avoid them, (4) scalability or performance considerations at 10M+ users, and (5) tooling and observability. Aim for 2–3 minutes of structured speaking with a STAR-format story for behavioural variants.

Code:

```
# Day 60 – Software Testing & QA
# Sample implementation pattern
def solution_60(data):
    """
    Production pattern for Software Testing & QA.
    Time: O(n log n) | Space: O(n)
    """
    # Step 1: Validate input
    if not data:
        return None
```

```
# Step 2: Core algorithm
result = sorted(data, key=lambda x: x) # adapt per problem
# Step 3: Post-process
return result

if __name__ == '__main__':
    print(solution_60([3,1,4,1,5,9])) # example
```

Section B — Python Coding Problems

Q1. Implement a production-grade solution for the classic Software Testing & QA coding problem.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 1 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

print(solve([2,7,9,3,1])) # 12
```

Q2. Write an optimised Python solution with $O(n \log n)$ or better complexity.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 2 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
```

```

dp[1] = nums[0]
for i in range(2, n+1):
    dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q3. Solve this problem using dynamic programming with memoisation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 60 Coding Problem 3 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12

```

Q4. Design a class hierarchy using OOP principles for a real-world Software Testing & QA system.

● **MED**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```

# Day 60 Coding Problem 4 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]

```

```
print(solve([2,7,9,3,1])) # 12
```

Q5. Implement a concurrent version of this algorithm using asyncio.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 5 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q6. Write comprehensive unit tests for the solution using pytest.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 6 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q7. Implement the solution using a functional programming approach (map/filter/reduce).

● MED

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 7 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q8. Optimise the naive $O(n^2)$ solution to $O(n)$ using a hash map.

● HARD

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 8 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q9. Solve using divide and conquer with a clear recurrence relation.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 9 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q10. Implement with generators to handle datasets too large for memory.

● **HARD**

Answer:

Approach: identify the pattern (sliding window / two-pointer / DP / graph traversal), pick the right data structure, implement with clear variable names, test edge cases (empty input, single element, all same, negatives).

Hint: Think about the optimal data structure for Software Testing & QA problems at this difficulty.

Code:

```
# Day 60 Coding Problem 10 – Software Testing & QA
def solve(nums):
    # Pattern: recognise subproblem structure
    n = len(nums)
    if n == 0: return 0
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n+1):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])
    return dp[n]
print(solve([2,7,9,3,1])) # 12
```

Q1. Design and query a database schema for a large-scale Software Testing & QA system.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q2. Write an optimised query with window functions to analyse Software Testing & QA metrics.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q3. Find anomalies in Software Testing & QA data using GROUP BY, HAVING and subqueries.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q4. Implement a reporting query with CTEs and recursive elements for Software Testing & QA.

● **MED**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,

```



```

SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q5. Write a query to calculate time-series trends for Software Testing & QA events.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q6. Design a schema supporting multi-tenant access control for a Software Testing & QA platform.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q7. Query to find the top-N performers in each category for Software Testing & QA data.

● MED

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q8. Write a query to detect data quality issues in a Software Testing & QA pipeline table.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
  SELECT *,
  RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
  FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;
```

Q9. Implement upsert (INSERT ON CONFLICT) for idempotent Software Testing & QA ETL.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```
-- Day 60 SQL – Software Testing & QA
WITH base AS (
  SELECT
    user_id,
    DATE_TRUNC('month', event_time) AS month,
    COUNT(*) AS events,
    SUM(value) AS total_value
  FROM events
  WHERE category = 'Software'
  GROUP BY 1, 2
),
ranked AS (
```

```

SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Q10. Query to compute a cohort retention analysis for Software Testing & QA users.

● **HARD**

Answer:

For Software Testing & QA SQL problems: always start by drawing the schema (entities, relationships, cardinalities). Identify whether you need window functions, CTEs, or subqueries. Check for NULLs, duplicates, and indexing. Explain query plan with EXPLAIN ANALYZE.

Code:

```

-- Day 60 SQL – Software Testing & QA
WITH base AS (
SELECT
user_id,
DATE_TRUNC('month', event_time) AS month,
COUNT(*) AS events,
SUM(value) AS total_value
FROM events
WHERE category = 'Software'
GROUP BY 1, 2
),
ranked AS (
SELECT *,
RANK() OVER (PARTITION BY month ORDER BY total_value DESC) AS rnk
FROM base
)
SELECT * FROM ranked WHERE rnk <= 10
ORDER BY month, rnk;

```

Section D — DSA Problems

Q1. Solve a hard Software Testing & QA DSA problem requiring advanced graph or DP techniques.

● **HARD**

Answer:

Identify subproblem: can you express the answer in terms of smaller inputs? Write the recurrence, add memoisation, convert to bottom-up DP. Analyse time and space complexity.

Hint: Think recursively, then iteratively with memoisation.

Code:

```

# Day 60 DSA 1 – Software Testing & QA
@functools.lru_cache(maxsize=None)

```

```
def dp(i, j):
    if i == 0 or j == 0: return 0
    if s1[i-1] == s2[j-1]:
        return 1 + dp(i-1, j-1)
    return max(dp(i-1, j), dp(i, j-1))
```

Q2. Implement an efficient Software Testing & QA algorithm with $O(n \log n)$ complexity.

● **HARD**

Answer:

Divide and conquer: split problem in half recursively, solve independently, combine in $O(n)$. Priority queue: maintain invariant of $O(\log n)$ insertions, $O(1)$ peek.

Hint: Divide and conquer or priority queue.

Code:

```
# Day 60 DSA 2 – Software Testing & QA
import heapq
def efficient_algo(data):
    heap = []
    for x in data:
        heapq.heappush(heap, x)
    return [heapq.heappop(heap) for _ in range(len(heap))]
```

Q3. Solve the Software Testing & QA variant of a classic backtracking problem.

● **HARD**

Answer:

Backtracking template: choose → explore → un-choose. Prune early when constraints are violated to cut search space from $O(n!)$ to manageable.

Hint: State space: what choices do you have at each step? When do you prune?

Code:

```
# Day 60 DSA 3 – Backtracking
def solve(choices, path, results):
    if is_solution(path):
        results.append(path[:])
        return
    for c in choices:
        if is_valid(c, path):
            path.append(c)
            solve(remaining(choices, c), path, results)
            path.pop()
```
